

安全护栏与审计参考资料库

本文档整理 `tmp_MCP` 安全意图校验器、审计日志、防篡改链路、规则配置化和 **Agent** 工具调用设计参考的标准、论文与开源仓库。它的用途是给开发提供设计依据，也方便比赛答辩时说明“创意来源不是拍脑袋，而是参考了成熟安全工程和 **Agent** 工具调用实践”。

1. MCP 与 Agent 安全

资料	链接	对本项目的启发
MCP Security Best Practices	https://modelcontextprotocol.io/specification/2025-06-18/basic/security_best_practices	MCP 工具调用必须考虑授权、用户同意、会话劫持、Token 透传、SSRF、最小暴露等风险；对应本项目的 <code>guardrail</code> 、审批和工具边界。
MCP Authorization	https://modelcontextprotocol.io/specification/2025-06-18/basic/authorization	为后续 B/S 架构、远程 MCP、审批用户身份和 <code>client consent</code> 预留依据。
OWASP Top 10 for LLM Applications	https://owasp.org/www-project-top-10-for-large-language-model-applications/	将 Prompt Injection、敏感信息泄露、过度代理等风险映射为规则类别和测试用例。
OWASP MCP Top 10	https://owasp.org/www-project-mcp-top-10/	针对 MCP 的工具投毒、上下文注入、过度权限等风险，为 <code>rules.yaml</code> 分类提供安全语义。
NIST AI Risk Management Framework	https://www.nist.gov/itl/ai-risk-management-framework	提供 AI 风险治理、度量、管理和持续改进框架，支撑 <code>low/medium/high/critical</code> 风险分级。

Table 1

2. 最小权限与运维 SOP

资料	链接	对本项目的启发
NIST SP 800-53 AC-6 Least Privilege	https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final	启发 <code>ExecutionActionTemplate</code> 中按动作声明推荐受限身份、允许范围和禁止范围。
Microsoft PowerShell Just Enough Administration	https://learn.microsoft.com/powershell/scripting/security/recommending/jea/overview	启发 Windows 写操作后续通过 JEA Endpoint 暴露受限服务控制、防火墙和 ACL 能力。
sudoers Manual	https://www.sudo.ws/docs/man/sudoers.man/	启发 Linux 后续使用命令 <code>allowlist</code> ，而不是让 MCP Server 直接以 root 运行。
Google SRE Book - Managing Incidents	https://sre.google/sre-book/managing-incidents/	启发 SOP 设计中的“先观测、再判断、再缓解、最后复盘”流程。
Google SRE Workbook - Incident Response	https://sre.google/workbook/incident-response/	启发将磁盘满、端口冲突、服务异常等场景整理为可重复执行的 <code>runbook</code> 。

Table 2

3. 链路追踪与溯源

资料	链接	对本项目的启发
W3C Trace Context	https://www.w3.org/TR/trace-context/	<code>trace_id</code> 使用 32 位十六进制字符串，便于未来接入 HTTP/B/S 链路追踪。
OpenTelemetry Traces	https://opentelemetry.io/docs/concepts/signals/traces/	将一次用户请求拆成多个 <code>span/event</code> 的思路，可用于后续展示 <code>guardrail</code> 、 <code>tool_result</code> 、 <code>audit_query</code> 时间线。
W3C PROV-DM	https://www.w3.org/TR/prov-dm/	用“实体、活动、代理”的方式表达 <code>provenance</code> ，可映射为用户意图、工具调用、安全决策和执行结果。

Table 3

4. 防篡改审计与供应链完整性

资料	链接	对本项目的启发
in-toto	https://in-toto.io/	强调步骤顺序、执行主体和产物完整性；启发审计日志记录“谁在何时执行了哪一步”。
in-toto GitHub	https://github.com/in-toto/in-toto	其 <code>link metadata</code> 和 <code>layout</code> 思路可借鉴到后续审批链、执行链和签名链。
Sigstore Rekor	https://github.com/sigstore/rekor	透明日志思想可作为本地哈希链之后的增强方向：将每日最终 <code>hash</code> 锚定到外部系统。
RFC 2104 HMAC	https://www.rfc-editor.org/rfc/rfc2104	启发本地审计锚点使用 HMAC-SHA256 证明锚点由持有密钥的一方生成。
Certificate Transparency RFC 6962	https://www.rfc-editor.org/rfc/rfc6962	启发后续把本地 <code>anchor</code> 迁移为真正 <code>append-only</code> 透明日志。

Table 4

5. 规则库与策略引擎

资料	链接	对本项目的启发
Open Policy Agent	https://www.openpolicyagent.org/	策略与业务代码分离，启发 <code>rules.yaml</code> 与 <code>risk_engine.py</code> 解耦。
OPA GitHub	https://github.com/open-policy-agent/opa	后续可评估是否用 Rego 表达复杂审批策略。
Sigma	https://github.com/SigmaHQ/sigma	安全规则采用 YAML、包含规则元数据、来源、描述、状态和测试生态，适合作为 <code>rules.yaml</code> 的结构参考。
YARA	https://github.com/VirusTotal/yara	启发面向模式匹配的规则组织方式，适合危险命令和注入特征匹配。
osquery	https://github.com/osquery/osquery	启发 OS 状态采集标准化，后续只读感知工具可逐步向结构化查询靠拢。

Table 5

6. Agent 工具调用论文

论文	链接	对本项目的启发
ReAct: Synergizing Reasoning and Acting in Language Models	https://arxiv.org/abs/2210.03629	将 reasoning、action、observation 交替组织；本项目把这些动作落成 <code>trace_id</code> 串联的审计事件。
Toolformer: Language Models Can Teach Themselves to Use Tools	https://arxiv.org/abs/2302.04761	说明模型使用工具会成为通用能力；本项目进一步强调工具调用必须受 deterministic guardrail 约束。

Table 6

7. 映射到当前设计

项目设计	参考来源	说明
<code>config/guardrails/rules.yaml</code>	Sigma、YARA、OPA、OWASP	将危险命令、Prompt Injection、敏感路径和工具风险从代码常量迁移为规则库。
<code>GuardrailDecision</code>	NIST AI RMF、OWASP	用风险等级和决策结果统一表达 <code>allow</code> 、 <code>require_approval</code> 、 <code>deny</code> 。
<code>prev_hash/event_hash</code>	in-toto、Rekor、透明日志思想	第一版实现本地哈希链，后续可将每日 <code>anchor</code> 外部签名或上传。
<code>anchors.jsonl</code> + HMAC	Rekor、RFC 2104、Certificate Transparency	将审计文件链尾 <code>hash</code> 和文件摘要锚定到独立账本，弥补整链重算风险。
<code>trace_id</code> 串联	W3C Trace Context、OpenTelemetry、PROV	把一次自然语言请求中的安全校验、工具结果和审计查询串成可回放链路。
<code>guard_context</code>	MCP Security、OWASP MCP	上游璇玑 Guardrail 作为外部证据输入，MCP 内部仍保持二次确定性校验。
<code>ExecutionActionTemplate</code>	NIST AC-6、PowerShell JEA、sudoers	将每类写操作绑定到固定模板、推荐受限身份、允许范围和回滚策略。

项目设计	参考来源	说明
ops_sops.py	Google SRE Runbook/Incident Response	将经典运维故障沉淀为只读 步骤、决策点、推荐写模板 和 AstrBot 测试口令。

Table 7

8. 后续研究方向

- 比较 YAML 规则、Rego 策略和 Python 规则在可解释性、性能、现场修改成本上的差异。
- 补充 Prompt Injection 攻击与防御综述，用于扩展中英文对抗样例库。
- 研究透明日志和远程审计锚定，增强本地文件被整体重写时的发现能力。
- 研究 OpenTelemetry 与 B/S 管理端结合，把 MCP 工具调用展示为可视化时间线。
- 研究基于 osquery 的只读资产与状态采集，增强 Linux/麒麟平台上的标准化观测能力。